

Overview: Artificial Intelligence (AI)

Source Summary: **Based on lecture notes, textbooks, and AI materials from IIT Jodhpur and Russell & Norvig's textbook.**

Definition:

Artificial Intelligence (AI) is the science and engineering of creating intelligent agents—systems that perceive their environment and take actions to maximize their chances of achieving goals.

Core Topics in AI:

1. Agents and Environments

- Agent: An entity that perceives its environment via sensors and acts through actuators.
- Types: Simple reflex, model-based, goal-based, utility-based, and learning agents.
- Environment types: Observable vs. partially observable, deterministic vs. stochastic, episodic vs. sequential, static vs. dynamic, discrete vs. continuous.

2. Problem Solving via Search

- Uninformed Search: DFS, BFS, UCS, IDS (no domain knowledge).
- Informed Search: A^* , Greedy Best-First (uses heuristics).
- Local Search: Hill Climbing, Simulated Annealing, Evolutionary Algorithms.
- CSP (Constraint Satisfaction Problems): Sudoku, N-Queens, Map Coloring.

3. Knowledge-Based Agents & Logic

- Use logic (e.g., propositional and first-order) to represent and infer knowledge.
- Inference Engine: Derives new knowledge using entailment.
- Declarative approach: Separates knowledge from procedure.

4. Learning from Examples (Machine Learning)

- Supervised Learning: Learn from labeled data.

- ML Models: Decision Trees, SVMs, Neural Networks.
- Deep Learning: Subset using multi-layer neural networks (CNNs, RNNs).

5. Applications of AI

- NLP: Chatbots, sentiment analysis.
 - Computer Vision: Object recognition, autonomous vehicles.
 - Robotics: Motion planning, autonomous agents.
 - Planning & Reasoning: Game playing, scheduling, diagnosis.
-

Modern Challenges & Scope

- Ethics & Fairness: Bias, accountability in AI systems.
 - Trust & Safety: Interpretability, robust decision-making.
 - Scalability: Large state/action spaces, real-time inference.
 - Future Prospects: AI market projected to exceed \$1.81 trillion by 2030.
-

✅ Final Takeaway:

AI is an interdisciplinary field encompassing logic, search, learning, knowledge representation, perception, and action. Its goal is to build systems that behave intelligently, and it spans applications from smart assistants to autonomous agents in complex real-world environments.

Artificial Intelligence Agent

Topic: **Intelligent Agent**

Source: **IIT Jodhpur Lectures + Russell & Norvig Textbook**

Definition:

An AI agent is an autonomous entity that perceives its environment through sensors and acts upon it through actuators to achieve specific goals.

Agent Structure:

According to Russell & Norvig, an agent is defined as a function:

Agent: $f : \text{Percept Sequences} \rightarrow \text{Actions}$

Components (PEAS Framework):

Used to specify the task environment of an intelligent agent:

Component	Description
P	Performance Measure (e.g., accuracy, time)
E	Environment (e.g., grid, web, real world)
A	Actuators (e.g., motors, API calls)
S	Sensors (e.g., cameras, file inputs)

Types of AI Agents:

Type	Description	Example
Simple Reflex Agent	Acts only on current percept (no memory)	Thermostat
Model-Based Agent	Maintains internal state	Vacuum robot
Goal-Based Agent	Chooses actions to reach a goal	Maze solver
Utility-Based Agent	Chooses actions to maximize utility	Path planner with cost optimization
Learning Agent	Improves performance over time	Chess-playing AI

Agent Environments:

Property	Types
Observability	Fully / Partially Observable

Property	Types
Determinism	Deterministic / Stochastic
Episodic	Episodic / Sequential
Dynamics	Static / Dynamic
Discreteness	Discrete / Continuous
Agents	Single-agent / Multi-agent

✔ Final Answer:

✔ An AI agent is a system that perceives its environment through sensors and acts upon it using actuators to achieve specific goals, possibly learning and adapting over time.

Solving Problems by Searching

Topic: **Problem-Solving Agents & Search Algorithms**

Source: **IIT Jodhpur Classnotes + Russell & Norvig Book**

Definition:

Search is a fundamental technique in AI to find a sequence of actions that leads from a start state to a goal state.

An AI problem-solving agent uses search algorithms to determine which actions to take by exploring a space of possible states.

Step-by-Step Process of Search:

Step 1: Formulate the Problem

- Initial state
- Goal state
- Actions
- Transition model
- Path cost

Step 2: Define the Search Space

- The search tree is a graph of all possible states reachable from the initial state using the available actions.

Step 3: Apply a Search Algorithm

- Use a suitable algorithm to explore the state space efficiently.
-

Types of Search Algorithms:

1. Uninformed Search (Blind Search):

(No domain-specific knowledge)

Algorithm	Properties
BFS (Breadth-First Search)	Complete, Optimal (if cost = 1), High memory

Algorithm	Properties
DFS (Depth-First Search)	Low memory, not complete, not optimal
UCS (Uniform Cost Search)	Complete, Optimal, like BFS but considers cost
IDS (Iterative Deepening Search)	Combines BFS optimality with DFS memory efficiency

2. Informed Search (Heuristic Search):

(Uses domain-specific knowledge)

Algorithm	Properties
Greedy Best-First Search	Uses $h(n)$, fast but not optimal
A*	Uses $f(n) = g(n) + h(n)$, complete and optimal (if h is admissible)

 Problem Definition Format:

$$\text{Problem} = \langle S, A, T, s_0, G, C \rangle$$

Where:

- S : Set of states
- A : Set of actions
- $T(s, a)$: Transition model
- s_0 : Initial state
- G : Goal test
- $C(s, a, s')$: Cost function

 Applications:

- Pathfinding (e.g., GPS)
- Puzzle solving (e.g., 8-puzzle, Sudoku)
- Game playing (e.g., Chess AI)
- Robotics planning (e.g., cleaning robot path)

✔ Final Answer:

✔ *Problem-solving by searching* is a systematic way in AI to find a solution (path of actions) by exploring possible states using search algorithms, either uninformed (like BFS/DFS) or informed (like A*/Greedy).

🔍 A* Search

Topic: Informed (Heuristic) Search
Type: Graph-based, Optimal Search

How It Works:

A* finds the least-cost path from an initial state to a goal using both:

- g(n): cost from start to node n
- h(n): estimated cost from n to goal (heuristic)

$$f(n) = g(n) + h(n)$$

Steps:

Step 1: Add the start node to a priority queue (based on f(n))
Step 2: Repeat:

- Remove node with lowest f(n)
- If it's the goal → done
- Otherwise, expand it (generate successors)
- Add successors to the queue

✔ Properties:

Property	A* Search
Completeness	✔ Yes (if branching factor is finite)

Property	A* Search
Optimality	✔ Yes (if <code>h(n)</code> is admissible)
Time Complexity	! High (can be exponential)
Space	! High (stores all explored nodes)

Example:

For a pathfinding problem on a map:

- `g(n)` = distance from start city
- `h(n)` = straight-line distance to goal
- A* uses both to find the shortest route

 Local Search

Topic: **Optimization**

Type: **State-based, Memory Efficient**

How It Works:

Local search starts with one state and iteratively moves to a neighbor with a better value (utility or cost), instead of building a full search tree.

Types of Local Search:

Algorithm	Description
Hill Climbing	Greedy ascent to best neighbor
Simulated Annealing	Allows bad moves to escape local maxima
Genetic Algorithms	Uses natural selection + crossover/mutation

 Properties:

Property	Local Search
Completeness	✗ Not guaranteed
Optimality	✗ Not guaranteed (gets stuck)
Time/Space	✓ Efficient (constant memory)
Usage	✓ Useful in huge state spaces

Example Use-Cases:

- N-Queens problem
- Traveling Salesman Problem (TSP)
- Neural Network Weight Tuning

 Comparison: A* vs Local Search

Feature	A*	Local Search
Search Space Explored	Expands full paths	Only considers current + neighbors
Memory Usage	High	Low
Optimality	Yes (with admissible h)	No (unless lucky)
Typical Use Case	Pathfinding, planning	Optimization problems

 Final Summary:

- A* = Accurate, optimal, but memory heavy. Best for pathfinding and deterministic planning.
- Local Search = Fast and lightweight. Best for large, continuous, or ill-defined state spaces.

 Hill Climbing vs Simulated Annealing

Topic: Local Search Algorithms

Context: Used for optimization problems in large or continuous search spaces

 Hill Climbing

Greedy local search that always moves to the neighbor with the highest value (or lowest cost).

 Steps:

- Step 1: Start with an initial solution
- Step 2: Evaluate neighboring solutions
- Step 3: Move to the best neighbor
- Step 4: Repeat until no improvement is possible

 Types of Hill Climbing:

Variant	Description
Simple	Always pick best neighbor
Stochastic	Pick a random better neighbor
First-Choice	Explore neighbors randomly until one is better
Random-Restart	Repeat HC from multiple initial points to avoid local maxima

 Problems with Hill Climbing:

Challenge	Description
Local maxima	Peaks that aren't the global maximum

Challenge	Description
Plateau	Flat regions with no gradient
Ridges	Path requires temporary decrease

✔ Best for:

- Simple optimization (e.g., N-Queens)
- When goal is to get a *good enough* solution fast

🔥 Simulated Annealing (SA)

Inspired by metallurgy: *occasionally allows worse moves* to escape local optima

✔ Steps:

- Step 1: Start with initial solution
- Step 2: Randomly choose a neighbor
- Step 3: If it's better → move
- Step 4: If worse → move with probability $P = e^{\{-\Delta E / T\}}$, where:
- ΔE = increase in cost
 - T = "temperature" that decreases over time (cooling schedule)

🧠 Intuition:

- High T → accepts bad moves easily (exploration)
- Low T → behaves more like hill climbing (exploitation)

📊 Parameters:

- Initial Temperature (T_0)
- Cooling Schedule: Exponential or logarithmic
- Stopping Criteria: Temperature low or max iterations reached

✓ Best for:

- Escape from local maxima
- Traveling Salesman Problem (TSP), circuit design, scheduling

📊 Comparison Table

Feature	Hill Climbing	Simulated Annealing
Strategy	Greedy	Probabilistic (accepts worse moves)
Escape local max?	✗ No	✓ Yes (via probability)
Global Optimum	Not guaranteed	Can reach (with right schedule)
Time Complexity	Low	Moderate
Memory	Constant	Constant

✓ Final Takeaway:

- Hill Climbing is fast but can get stuck in local optima.
- Simulated Annealing trades speed for flexibility—better at escaping traps and exploring the space.

✳ First-Order Logic (FOL)

Topic: Knowledge Representation & Reasoning

Type: Symbolic Logic for AI Agents

Source: IIT Jodhpur Lecture + Russell & Norvig

First-Order Logic (FOL), also called predicate logic, is a formal system used to represent facts about the world and perform logical reasoning over them.

Unlike propositional logic, which handles simple true/false statements, FOL supports:

- Objects
- Relations (predicates)
- Quantifiers ($\forall$, $\exists$)

 Components of FOL:

Component	Example	Meaning
Constants	John , 4	Specific objects
Variables	x , y , z	Generic placeholders
Predicates	Likes(John, IceCream)	Relationships between objects
Functions	FatherOf(x)	Maps input to another object
Quantifiers	$\forall$, $\exists$	Universal / Existential
Connectives	$\wedge$, $\vee$, $\rightarrow$, $\neg$	Logical AND, OR, IMPLIES, NOT

 Example Sentences in FOL:

1. Everyone loves Ice Cream

$$\forall x \text{ Loves}(x, \text{IceCream})$$

2. There exists someone who likes chocolate:

$$\exists x \text{ Likes}(x, \text{Chocolate})$$

3. If a person is a father, then he is male:

$$\forall x \text{ Father}(x) \rightarrow \text{Male}(x)$$

 Semantics (Meaning):

- A model assigns meaning to constants, predicates, and functions.
- A sentence is true in a model if it holds given those assignments.

- Inference uses rules to derive new truths (e.g., Modus Ponens).

 Use in AI Agents:

FOL helps knowledge-based agents:

- Represent structured environments (e.g., family trees, maps)
- Perform logical inference (e.g., Prolog)
- Support reasoning in expert systems, NLP, planning, etc.

 FOL vs. Propositional Logic:

Feature	Propositional Logic	First-Order Logic
Expressiveness	Low	High
Variables	✗ No	✓ Yes
Quantifiers	✗ No	✓ Yes
Relationships	No	Yes (via predicates)

 Final Answer:

✓ *First-Order Logic (FOL)* is a powerful logical language used in AI to represent and reason about structured knowledge involving objects, relations, and quantifiers, making it far more expressive than propositional logic.

 Constraint Satisfaction Problems (CSPs)

Topic: **Search & Reasoning in AI**

Type: **Factored Representation Problems**

Source: **Lecture Notes + Russell & Norvig**

What is a CSP?

A Constraint Satisfaction Problem (CSP) is a type of problem where:

- You have a set of variables,
- Each with a domain of values,
- And a set of constraints that restrict the values the variables can simultaneously take.

CSP Structure:

A CSP is defined as a triple:

$$CSP = (X, D, C)$$

Symbol	Meaning
X	Set of variables: {X ₁ , X ₂ , ..., X _n }
D	Domains for each variable: D(X ₁), D(X ₂)...
C	Constraints (relations over variables)

Goal:

Find an assignment of values to all variables such that all constraints are satisfied.

Examples of CSPs:

Problem	Variables	Domain	Constraints
Map Coloring	Regions (WA, NT...)	Colors {Red, Green, Blue}	Adjacent regions ≠ same color
N-Queens	Queens' columns	Row positions {1..N}	No two queens in same row/column/diagonal
Sudoku	Grid cells	Numbers {1..9}	AllDiff in rows, cols, boxes
Job Scheduling	Jobs	Time slots	No overlapping times

🧠 Types of Constraints:

Type	Example
Unary	$X \neq 3$
Binary	$X \neq Y, X < Y$
Higher-order	$\text{AllDiff}(X, Y, Z) = \text{all values differ}$

🔧 Solving CSPs:

1. Backtracking Search (**baseline**)
 - Assign variables one by one
 - Backtrack when constraints are violated
2. Constraint Propagation
 - Forward Checking: eliminate incompatible values from domains
 - Arc Consistency (AC-3): revise domains so binary constraints are satisfied
3. Heuristics for Efficiency
 - Minimum Remaining Values (MRV): pick variable with fewest legal values
 - Degree Heuristic: pick variable involved in most constraints
 - Least Constraining Value (LCV): pick value that rules out the fewest options
4. Local Search (for large CSPs)
 - e.g., Min-Conflicts heuristic (used for N-Queens)

📊 Why CSPs Matter in AI:

- Efficient, domain-independent solvers exist
- Applied in scheduling, planning, vision, natural language, etc.
- Represent factored state space (vs. atomic in classical search)

✅ Final Answer:

✓ *Constraint Satisfaction Problems (CSPs)* are a structured way to model problems with variables and rules, aiming to find assignments that satisfy all constraints. They allow efficient reasoning and are widely used in AI planning, puzzles, scheduling, and logic inference.

Adversarial Search and Games

Topic: **Game Playing in AI**

Source: **Russell & Norvig + IIT Jodhpur notes**

What is Adversarial Search?

Adversarial search deals with multi-agent environments, where agents compete rather than cooperate.

Typical use case: Two-player turn-based games like Chess, Tic-Tac-Toe, Go, etc.

Here, the opponent is not just a passive environment—but an intelligent adversary trying to defeat the agent.

Objective:

Design an agent that chooses optimal moves by anticipating opponent's responses.

Game Model Elements:

Component	Description
<code>s</code> (State)	Current game configuration
<code>A(s)</code>	Set of legal actions in state <code>s</code>
<code>T(s, a)</code>	Transition function next state
Players	Typically MAX vs MIN (e.g., you vs AI)
Terminal Test	Is the game over?
Utility(s)	Final score of game (win/loss/tie etc.)

Minimax Algorithm

Strategy assuming perfect play from both players.

Steps:

1. Build game tree from current state.
2. Alternate levels between:
 - MAX: chooses the move with maximum utility.
 - MIN: opponent chooses move with minimum utility.
3. Backpropagate values from terminal states to root using:

$$\text{Minimax}(s) = \begin{cases} \max_{a \in A(s)} \text{Minimax}(T(s, a)) & \text{if MAX's turn} \\ \min_{a \in A(s)} \text{Minimax}(T(s, a)) & \text{if MIN's turn} \end{cases}$$

Example: Tic-Tac-Toe

- Each leaf has a utility: Win = +1, Loss = -1, Draw = 0
 - MAX tries to maximize this; MIN tries to minimize it
-

Alpha-Beta Pruning

Optimization of minimax—cuts off branches that don't affect the final decision.

How it works:

- α (alpha): best value MAX can guarantee
- β (beta): best value MIN can guarantee

If a node is worse than the best option already found, stop exploring it.

 Same result as minimax but faster

Complexity:

Algorithm	Time Complexity	Space
Minimax	$O(b^m)$	$O(bm)$
Alpha-Beta	$O(b^{m/2})$ (ideal)	$O(bm)$

- **b** = branching factor
- **m** = maximum depth of tree

Games with Chance:

- Example: Backgammon
- Use Expectiminimax: combine minimax with chance nodes

Evaluation Functions (for non-terminal nodes)

- Used to estimate utility when game tree is too deep
- e.g., number of pieces, board control, threats

Final Answer:

 *Adversarial search* is used in competitive multi-agent settings (like games), where agents must anticipate opponents' actions. It uses strategies like Minimax and Alpha-Beta pruning to make optimal decisions under uncertainty and time constraints.

Markov Decision Process (MDP)

Topic: **Sequential Decision Making**

Used In: **Planning, Reinforcement Learning**

What is an MDP?

An MDP is a formal model used to describe environments where outcomes are partly random and partly under the control of the agent.

It extends the idea of search to include uncertainty and rewards.

 Components of an MDP:

$$\text{MDP} = (S, A, T, R, \gamma)$$

Symbol	Meaning
S	Set of states
A	Set of actions
T(s,a,s')	Transition model: probability of moving to s' from s using a
R(s)	Reward function: immediate reward for being in s
γ	Discount factor (0 ≤ γ ≤ 1), weights future rewards

 Goal:

Find a policy $\pi(s)$ = best action to take in state s to maximize expected cumulative reward.

 Key Equations:

- Value Function (expected reward from s):

$$V^{\pi}(s) = R(s) + \gamma \sum_{s'} T(s, \pi(s), s') V^{\pi}(s')$$

- Bellman Optimality Equation:

$$V^*(s) = \max_a \left[R(s) + \gamma \sum_{s'} T(s, a, s') V^*(s') \right]$$

 Solving MDPs:

- Value Iteration: Repeatedly update value function using Bellman equation

- Policy Iteration: Alternate between policy evaluation and improvement



Reinforcement Learning (RL)

Topic: **Learning from Experience**

Used In: **Game AI, Robotics, Adaptive Systems**



What is RL?

RL is when an agent learns to act optimally in an environment by trial and error, receiving rewards/punishments.



Key Elements:

Component	Description
Agent	Learner/decision-maker
Environment	Where the agent operates
State (s)	Current situation
Action (a)	Choices available
Reward (r)	Feedback signal
Policy (π)	Strategy for choosing actions
Value Function (V, Q)	Predicts future reward



Types of RL:

Type	Description
Model-Free	Learns directly from experience (e.g., Q-Learning, SARSA)
Model-Based	Learns a model of the environment (T and R)
On-policy	Learns from actions taken (e.g., SARSA)

Type	Description
Off-policy	Learns from optimal actions (e.g., Q-Learning)

✓ Popular Algorithm: Q-Learning

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

 Bayesian Networks (Bayes Nets)

Topic: Probabilistic Reasoning

Used In: Medical Diagnosis, NLP, Robotics, Spam Filters

 What is a Bayesian Network?

A Bayesian Network is a directed acyclic graph (DAG) where:

- Nodes = random variables
- Edges = conditional dependencies
- Each node has a Conditional Probability Table (CPT)

 Example:

text

Cough Cold Fever

- Cold causes both Cough and Fever
- We can reason: P(Cough | Cold) and compute probabilities efficiently

✓ Key Concepts:

Term	Meaning
Prior	Initial belief before evidence
Likelihood	How likely evidence is given a cause

Term	Meaning
Posterior	Updated belief after seeing evidence
Conditional Independence	Reduces computation

 Bayes' Rule:

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)}$$

Used for diagnostic inference: from evidence → cause

 Inference Methods:

- Exact Inference: Variable elimination
- Approximate: Sampling (e.g., likelihood weighting, Gibbs sampling)

 Final Summary Table:

Concept	MDP	RL	Bayesian Network
Type	Sequential decision model	Learning from interaction	Probabilistic graphical model
Goal	Optimal policy	Learn optimal policy from trial	Compute beliefs from evidence
Uncertainty	In transitions	In feedback and environment	In variable dependencies
Learning Required?	Not always (planning)	Yes (exploration/exploitation)	No (requires known probabilities)
Example Use	Robot path planning	Game-playing agent	Disease diagnosis, spam filtering

-
-
-

Absolutely! Let's explore:

Automated Planning

Topic: **Intelligent Agent Decision-Making**

Used In: **Robotics, Game AI, Logistics, Scheduling**

Source: **Russell & Norvig + IIT Jodhpur classnotes**

What is Automated Planning?

Automated Planning is the area of AI that deals with formulating a sequence of actions that leads an agent from an initial state to a goal state, without human intervention.

Key Characteristics:

- Planning is deliberative: it simulates future possibilities before acting.
- Agent must search for an action sequence to satisfy its goals.
- Focuses on goal-based agents in deterministic, fully observable, and discrete environments.

Planning Problem Representation:

A planning problem is formally defined as:

$$P = \langle S, A, T, s_0, G \rangle$$

Component	Description
S	Set of world states
A	Set of possible actions
T(s, a)	Transition function: result of doing a in s

Component	Description
s_0	Initial state
G	Goal condition(s)

🧠 Classical Planning Assumptions:

1. Deterministic actions (known effects)
2. Fully observable environment
3. Static world (doesn't change during planning)
4. Instantaneous actions
5. Known initial state

↺ Types of Planning:

Type	Description
State-space search	Search from initial state to goal state
Plan-space planning	Search over partial plans
Hierarchical Planning (HTN)	Uses abstract high-level actions
Conditional Planning	For uncertain outcomes
Temporal Planning	Considers action durations and time

🔧 Planning Algorithms:

1. Forward Search (Progression Planning)

- Start from s_0 , apply actions to reach goal.
- Each node is a world state.

2. Backward Search (Regression Planning)

- Start from goal, find actions that could lead to it.
- Each node is a goal condition.

3. Partial Order Planning (POP)

- Construct plans without fixing order too early
- Useful when actions are loosely dependent

STRIPS Representation:

Standard planning language (used in classical planners)

Each action has:

- Preconditions: What must be true to apply action
- Add list: What becomes true after action
- Delete list: What becomes false after action

Example:

text

Action: Move(A, B)
Preconditions: At(A)
Add: At(B)
Delete: At(A)

Planning in Practice:

Domain	Use Case
Robotics	Motion & task planning
Game AI	NPC strategy & story sequencing
Logistics	Route and cargo scheduling
Web services	Workflow orchestration

Final Summary:

✔ *Automated Planning* enables AI agents to generate sequences of actions to achieve defined goals. It uses techniques like forward/backward search, partial order planning, and STRIPS-based action representations to solve complex, goal-driven problems in deterministic environments.

-
- A?
-